Supermarket Management System

Frontend–Backend–Database Mapping

Team work distribution and field-by-field database alignment verification

Member 1: Sales, Transactions & POS

Frontend Component	UI Features (Display, Input & Actions)	Backend Service & Controller (Spring Boot)	Database Objects & Tables (db_structure.sql)

PosComponent
`/pos`
	• Barcode / SKU product search
	• Real-time stock availability check
	• Multi-tier cart checkout
	• LankaPay QR / Cash / Card selection
	• Order Lookup & Cancel modal
		`OrderController.checkout()`
		`OrderController.cancelOrder()`
		`OrderService.processOrder()`
		`OrderDao.callPlaceOrder()`
		`OrderDao.callCancelOrder()`
			• `sp_place_order`: Atomic transaction (`FOR UPDATE`, JSON items, rolls back on `SQLEXCEPTION`)
			• `sp_cancel_order`: Atomic cancellation & stock restore
			• `trg_orderitem_check_stock`: BEFORE INSERT stock check
			• `trg_orderitem_reduce_stock`: AFTER INSERT stock deduction
			• **Clustered PK**: `order_item(order_id, product_id)`
			• Tables: `sales_order`, `order_item`, `payment`

ReceiptModalComponent	• Displays completed order receipt • Line items breakdown & line totals • VAT tax amount & net total • Cashier name & Branch info	`OrderController.getOrder()` `OrderDao.getOrderById()`	• `fn_order_total(order_id)`: Calculates order total • Generated column: `order_item.line_total` (`quantity * unit_price` STORED) • Tables: `sales_order`, `order_item`, `payment`

Member 2: Inventory, Supply Chain & Stock Flow

Frontend Component	UI Features (Display, Input & Actions)	Backend Service & Controller (Spring Boot)	Database Objects & Tables (db_structure.sql)

InventoryListComponent
`/inventory`
	• Multi-branch stock inventory table
	• Low-stock alert banner
	• Direct Restock modal (`sp_add_stock`)
	• Inter-branch stock transfer modal (`sp_transfer_stock`)
	• Purchase Orders & Delivery Receiving tab (`sp_receive_purchase`)
		`InventoryController.getBranchStock()`
		`InventoryController.restock()`
		`InventoryController.transferStock()`
		`PurchaseController.getOrders()`
		`PurchaseController.receiveDelivery()`
		`InventoryDao`, `SupplierDao`
			• `sp_add_stock`: `ON DUPLICATE KEY UPDATE` stock upsert
			• `sp_transfer_stock`: Atomic Branch A → Branch B transfer with `version` check
			• `sp_receive_purchase`: Receives supplier PO and auto-increments branch inventory
			• `trg_inventory_low_stock`: Fires when stock falls below `reorder_level`
			• `fn_stock_level(branch_id, product_id)`
			• `v_low_stock` & `ev_weekly_low_stock_scan`
			• Tables: `inventory`, `supplier`, `purchase_order`, `purchase_order_item`, `low_stock_alert`

Member 3: Products, Customers & Promotional Rules

Frontend Component	UI Features (Display, Input & Actions)	Backend Service & Controller (Spring Boot)	Database Objects & Tables (db_structure.sql)

ProductListComponent
`/products`
	• Product catalog browsing & category filter
	• Keyword search simulation via `FULLTEXT`
	• Update Product Price modal
	• Add New Product modal
		`ProductController.getProducts()`
		`ProductController.updatePrice()`
		`ProductController.saveProduct()`
		`ProductDao.saveProduct()`
		`ProductDao.updatePrice()`
			• `sp_save_product`: Product catalog upsert
			• `trg_product_price_audit`: Logs old vs new price into `price_audit_log`
			• **Index**: `FULLTEXT idx_product_ft(name)`
			• Tables: `product`, `category`, `price_audit_log`

CustomerListComponent
`/customers`
	• Customer loyalty directory
	• Tier classification cards (`GOLD`, `SILVER`, `BRONZE`)
	• Phone lookup search bar (`idx_customer_phone`)
	• Register Customer modal
	• Order history inspection modal
		`CustomerController.getCustomers()`
		`CustomerController.register()`
		`CustomerController.getCustomerOrders()`
		`CustomerDao.registerCustomer()`
			• `sp_register_customer`: Deduplicates phone & inserts customer
			• `fn_customer_tier(customer_id)`: Computes tier from cumulative spend / points
			• `v_customer_orders`: Order history view
			• **Index**: `idx_customer_phone`
			• Table: `customer`

DiscountListComponent
`/discounts`
	• Active promotional rules table (priority order)
	• Coupon manager (max uses, used count, expires)
	• Holiday calendar table
	• Create Coupon modal
		`DiscountController.getActiveRules()`
		`CouponController.getCoupons()`
		`CouponController.createCoupon()`
		`DiscountDao.applyDiscounts()`
			• `sp_apply_discounts`: SQL Cursor iterating active rules, enforcing 50% max order cap
			• `trg_rule_check`: Validates rule constraints (value $≤ 100%$, valid dates)
			• `fn_is_holiday(date)`
			• `fn_coupon_valid(code, rule_id)`
			• `v_active_discounts` & `ev_expire_discount_rules`
			• Tables: `discount_rule`, `coupon`, `holiday`, `order_discount`

Member 4: Reporting, Analytics & Big Data Performance

Frontend Component	UI Features (Display, Input & Actions)	Backend Service & Controller (Spring Boot)	Database Objects & Tables (db_structure.sql)

ReportsDashboardComponent
`/reports`
	• Executive KPI cards (Today's revenue, orders, discounts)
	• 7-Day & 30-Day Moving Averages
	• Product Ranking Leaderboard (`RANK() OVER`)
	• Daily Sales Summary precomputed table
	• **Monthly Branch Rollup Tab (`WITH ROLLUP`)**
	• **Category Hierarchical CTE Tab**
	• Promotional Impact Analysis table
	• 50k+ Rows `EXPLAIN ANALYZE` proof card
		`ReportController.getKPIs()`
		`ReportController.getDailySales()`
		`ReportController.getProductRankings()`
		`ReportController.getMonthlyRollup()`
		`ReportController.getCategoryCte()`
		`ReportDao.getMonthlyReport()`
		`ReportDao.getCategorySalesCte()`
			• `sp_monthly_report`: `GROUP BY branch_id, order_date WITH ROLLUP` (branch subtotals + chain grand total)
			• `sp_category_sales_cte`: Hierarchical recursive CTE product sales
			• `fn_avg_daily_sales(branch_id, days)`: Moving average
			• `v_daily_sales` (populated by `ev_daily_sales_summary`)
			• `v_product_ranking`: Window function `RANK() OVER (ORDER BY revenue DESC)`
			• `v_discount_impact`: Financial breakdown of surrendered discount margin
			• **Partitioning & Index**: RANGE partitioned `stock_audit_log`, `idx_order_branch_date`
			• Tables: `daily_sales_summary`, `stock_audit_log`

Member 5: Security, RBAC, Data Masking & Administration

Frontend Component	UI Features (Display, Input & Actions)	Backend Service & Controller (Spring Boot)	Database Objects & Tables (db_structure.sql)

LoginComponent
`/login`
	• System user authentication
	• 1-Click Viva Demo Role Switcher (`Floyd`, `Arlene`, `Kavindu`)
	• Real-time password strength meter (`fn_check_password_strength`)
	• Database session & role confirmation
		`AuthController.login()`
		`AuthService.authenticate()`
		`AuthDao.getLoginCredentials()`
			• `sp_get_login`: Retrieves BCrypt hash for username
			• `fn_check_password_strength(password)`: Validates password complexity directly in SQL
			• **MySQL 8 Roles**: `cashier_role`, `manager_role`, `admin_role`
			• Dedicated DB Users: `app_user`, `report_user`, `etl_user`, `admin_user`
			• Table: `employee`

AdminDashboardComponent
`/admin`
	• Employee directory table
	• **Register New Employee Modal** (provisions staff user, enforces unique username)
	• Change Role modal (`sp_change_user_role`)
	• Safe Deactivate button (`sp_deactivate_employee`) with self-lockout test
	• Reactivate button
	• Dynamic Data Masking toggle & table
	• Live Trigger Audit Logs table (`trg_employee_audit`)
	• MySQL 8 RBAC matrix
		`AuditController.getEmployees()`
		`AuditController.registerEmployee()`
		`AuditController.changeRole()`
		`AuditController.deactivateEmployee()`
		`AuditController.getAuditLogs()`
		`AuditDao`, `AuthDao`
			• `sp_change_user_role`: Updates role and logs into `security_event_log`
			• `sp_deactivate_employee`: Safely deactivates with self-lockout guard
			• `trg_employee_audit`: AFTER UPDATE on `employee` writing old/new role/branch to `employee_audit_log`
			• `trg_security_alert`: Prevents unauthorized modifications
			• `fn_mask_phone(phone)`: Masks customer phone (`077****567`) for cashiers
			• `v_masked_customer_directory` & `v_security_audit_log`
			• Tables: `employee`, `branch`, `employee_audit_log`, `security_event_log`

2. Field-by-Field Database Alignment Verification

We conducted an audit checking every input field, table column, and modal in the frontend against `docs/db_structure.sql`:

Frontend Entity / Model	Frontend Fields Displayed / Edited	Corresponding Database Column / Expression	Verification Result

Product	`id`, `name`, `sku`, `unitPrice`, `costPrice`, `categoryId`, `supplierId`, `imageUrl`, `dietType`, `isActive`	`product.product_id`
		`product.name`
		`product.sku`
		`product.unit_price`
		`product.cost_price`
		`product.category_id`
		`product.supplier_id`
		`product.image_url`
		`product.diet_type`
		`product.is_active`
			Exact Column Match

Category	`id`, `name`, `description`, `icon`, `itemCount`	`category.category_id`
		`category.name`
		`category.description`
		`category.icon`
		`COUNT(product_id)` (Aggregate query)
			Exact Column Match

Frontend Entity / Model	Frontend Fields Displayed / Edited	Corresponding Database Column / Expression	Verification Result

Customer	`id`, `fullName`, `phone`, `email`, `loyaltyPoints`, `tier`, `registeredDate`	`customer.customer_id`
		`customer.full_name`
		`customer.phone` (or `fn_mask_phone`)
		`customer.email`
		`customer.loyalty_points`
		`fn_customer_tier(customer_id)` (Scalar UDF)
		`customer.registered_date`
			Exact Column / UDF Match

SalesOrder	`orderId`, `branchId`, `customerId`, `employeeId`, `orderDate`, `status`, `subtotal`, `discountTotal`, `totalAmount`, `paymentMethod`	`sales_order.order_id`
		`sales_order.branch_id`
		`sales_order.customer_id`
		`sales_order.employee_id`
		`sales_order.order_date`
		`sales_order.status`
		`sales_order.subtotal`
		`sales_order.discount_total`
		`sales_order.total_amount`
		`payment.method ENUM('CASH', 'CARD', 'QR', 'ONLINE')`
			Exact Column Match

OrderItem	`productId`, `quantity`, `unitPrice`, `lineTotal`	`order_item.product_id`
		`order_item.quantity`
		`order_item.unit_price`
		`order_item.line_total` (STORED generated column)
			Exact Column Match

DiscountRule	`ruleId`, `name`, `ruleType`, `kind`, `value`, `minOrderAmount`, `maxDiscount`, `tier`, `startDate`, `endDate`, `priority`, `stackable`, `isActive`	`discount_rule.rule_id`
		`discount_rule.name`
		`discount_rule.rule_type`
		`discount_rule.kind`
		`discount_rule.value`
		`discount_rule.min_order_amount`
		`discount_rule.max_discount`
		`discount_rule.tier`
		`discount_rule.start_date`
		`discount_rule.end_date`
		`discount_rule.priority`
		`discount_rule.stackable`
		`discount_rule.is_active`
			Exact Column Match

Coupon	`code`, `ruleId`, `maxUses`, `usedCount`, `expiresOn`	`coupon.code`
		`coupon.rule_id`
		`coupon.max_uses`
		`coupon.used_count`
		`coupon.expires_on`
			Exact Column Match

Holiday	`holidayDate`, `name`	`holiday.holiday_date` `holiday.name`	**Exact Column Match**

Frontend Entity / Model	Frontend Fields Displayed / Edited	Corresponding Database Column / Expression	Verification Result

InventoryItem	`branchId`, `productId`, `quantity`, `reorderLevel`, `version`, `isLowStock`	`inventory.branch_id`
		`inventory.product_id`
		`inventory.quantity`
		`inventory.reorder_level`
		`inventory.version`
		`v_low_stock` (View match)
			Exact Column Match

PurchaseOrder	`poId`, `supplierId`, `branchId`, `orderDate`, `status`, `totalCost`	`purchase_order.po_id`
		`purchase_order.supplier_id`
		`purchase_order.branch_id`
		`purchase_order.order_date`
		`purchase_order.status`
		`purchase_order.total_cost`
			Exact Column Match

Employee	`employeeId`, `branchId`, `fullName`, `role`, `username`, `hireDate`, `isActive`	`employee.employee_id`
		`employee.branch_id`
		`employee.full_name`
		`employee.role`
		`employee.username`
		`employee.hire_date`
		`employee.is_active`
			Exact Column Match

EmployeeAuditLog	`logId`, `employeeId`, `action`, `oldValue`, `newValue`, `modifiedBy`, `modifiedAt`	`employee_audit_log.log_id`
		`employee_audit_log.employee_id`
		`employee_audit_log.old_role` / `new_role`
		`employee_audit_log.changed_by`
		`employee_audit_log.changed_at`
			Exact Column Match

SecurityEventLog	`logId`, `eventType`, `description`, `username`, `ipAddress`, `createdAt`	`security_event_log.log_id`
		`security_event_log.event_type`
		`security_event_log.details`
		`security_event_log.performed_by`
		`security_event_log.logged_at`
			Exact Column Match

Conclusion

- Every single screen in the Angular frontend corresponds directly to an assigned backend function and database object in `ADBMS_Team_Work_Distribution.md`.
- **Zero unsupported fields**: There are no disconnected UI elements; every data point either directly edits a column in MySQL InnoDB, queries an indexed view, or invokes a stored procedure or UDF.
- Per your instructions, the **backend directory remains untouched** until you authorize it.